Lesson 2: Component Characteristics and Models

Purpose: Establish what a software component is, how it is structured, and how to identify components in a system.

What makes something a component? → characteristics, boundaries, models, composition

Chapter 1 → CBSE foundations
Chapter 2 → Understanding and identifying components

2.1 Software Component Fundamentals

Definition of a software component

A software component is a **modular, encapsulated, and independently deployable unit of software that fulfils a distinct function and communicates through well-defined interfaces.**

Each word in the definition carries meaning:

Key term	What it means	Kenyan example
Modular	A separate, self-contained piece of a larger system	The Lipa na M-Pesa module is separate from Send Money
Encapsulated	Its internal data and logic are hidden and protected	You never see the PIN verification code
Independently deployable	It can be installed, updated or replaced on its own	Safaricom can update the Fuliza module without reissuing SIM cards
Distinct function	It does one clearly defined job	Buy Airtime only buys airtime
Well-defined interfaces	Others use it only through a published contract	Menu options, USSD codes, API endpoints

Real-world analogy: the M-Pesa SIM Toolkit (STK). Think of a software component like the M-Pesa SIM Toolkit applet on your phone. You do not need to know how Safaricom's servers process transactions; you interact with a clean menu interface (Send Money, Lipa na M-Pesa, Withdraw Cash). The applet encapsulates all encryption, network protocol details and security rules inside its boundary.

> **Note:** A component is not the same as a class. A component may be a single class, but more often it is a group of classes packaged and delivered together (for example a .dll, a NuGet package or a web service) and used through an interface.

Component responsibilities

A component's responsibility is its job description within a system. There are two kinds.

- **Functional responsibilities:** the calculations, business logic or data manipulation the component performs. Example: calculating 16% VAT on an invoice.

- **State responsibilities:** the data or state the component holds and protects. Example: maintaining the current user session or the items in a shopping cart.

A good component has **high cohesion**: all its responsibilities belong together. A component that calculates VAT, sends SMS and prints receipts has low cohesion and is hard to maintain. This is the Single Responsibility Principle applied at component level: *one component, one reason to change.*

C# example: functional responsibility

```
public class VatCalculator
{
    private const decimal VatRate = 0.16m;   // Kenya standard VAT rate

    public decimal CalculateVat(decimal netAmount)
    {
        if (netAmount < 0)
            throw new ArgumentException("Amount cannot be negative.");
        return Math.Round(netAmount * VatRate, 2);
    }

    public decimal GrossAmount(decimal netAmount) => netAmount + CalculateVat(netAmount);
}
```

Code explained. `VatCalculator` has a purely **functional** responsibility: it holds no customer data, it only computes. The VAT rate is a `private const`, so no client can change it. `Math.Round(..., 2)` rounds to cents. `GrossAmount` uses an expression-bodied member (`=>`) to return net plus VAT. For an invoice of KES 10,000, `CalculateVat` returns 1,600 and `GrossAmount` returns 11,600.

C# example: state responsibility

```
public class ShoppingCart
{
    private readonly Dictionary<string, int> items = new();   // hidden state

    public void AddItem(string productCode, int quantity)
    {
        if (quantity <= 0) throw new ArgumentException("Quantity must be positive.");
        items[productCode] = items.GetValueOrDefault(productCode) + quantity;
    }

    public void RemoveItem(string productCode) => items.Remove(productCode);

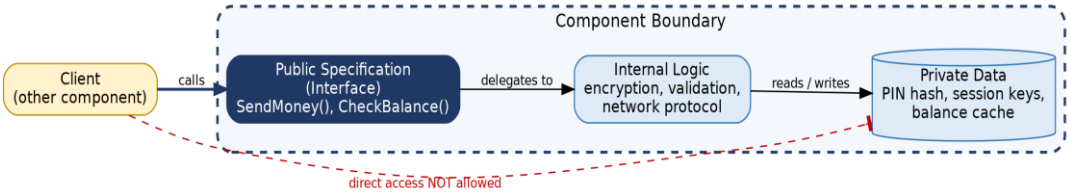
    public int ItemCount => items.Values.Sum();
}
```

Code explained. `ShoppingCart` has a **state** responsibility: it remembers what the customer has picked. The dictionary is private, so the only way to change the cart is through `AddItem` and `RemoveItem`, which enforce the rules. `GetValueOrDefault` returns 0 if the product is not yet in the cart. `ItemCount` exposes a read-only summary of the state (`Sum()` needs `using System.Linq;`).

Component boundaries

A component boundary strictly separates the **internal implementation** (hidden details) from the **external contract** (what the component exposes).

- Nothing outside the boundary can directly access private variables or internal helper methods.
- External interaction happens only through exposed methods or endpoints.



The component boundary: clients reach the internals only through the public specification

In C#, boundaries are enforced by access modifiers at two levels:

Level	Modifier	Effect
Class boundary	`private`	Visible only inside the class
Component (assembly) boundary	`internal`	Visible to classes inside the same .dll, hidden from other components
Public contract	`public`	Visible to every client

```
// Inside MpesaToolkit.dll
public interface IMpesaMenu                  // crosses the boundary
{
    bool SendMoney(string toPhone, decimal amount, string pin);
}

internal class PinVerifier                   // hidden from other components
{
    internal bool Verify(string pin) => pin.Length == 4;   // simplified
}

public class MpesaMenu : IMpesaMenu
{
    private readonly PinVerifier verifier = new();   // internal helper
    private decimal balance = 5000m;                  // private data

    public bool SendMoney(string toPhone, decimal amount, string pin)
    {
        if (!verifier.Verify(pin) || amount > balance) return false;
        balance -= amount;
        return true;
    }
}
```

Code explained. Only `IMpesaMenu` and `MpesaMenu` are `public`, so they form the external contract. `PinVerifier` is `internal`: other classes inside MpesaToolkit.dll can use it, but a separate component (for example a shop's app) cannot even see it. `balance` is `private` to the class. A client can only call `SendMoney`; it cannot read the balance directly or skip the PIN check.

Component specification vs implementation

- **Specification (the "what"):** the contract: inputs accepted, outputs returned, exceptions thrown, and pre- and post-conditions.
- **Implementation (the "how"):** the actual code, database queries and algorithms inside the component.

Pre-condition: what must be true before a call (the caller's duty). **Post-condition:** what the component guarantees after a successful call (the component's duty). Together they are called **design by contract**.

C# example: specification

```
/// <summary>Specification of a VAT service.</summary>
public interface IVatService
{
    /// <param name="netAmount">Pre-condition: netAmount >= 0.</param>
    /// <returns>Post-condition: VAT rounded to 2 decimal places, never negative.</returns
>
    /// <exception cref="ArgumentException">Thrown if netAmount is negative.</exception>
    decimal CalculateVat(decimal netAmount);
}
```

C# example: implementation

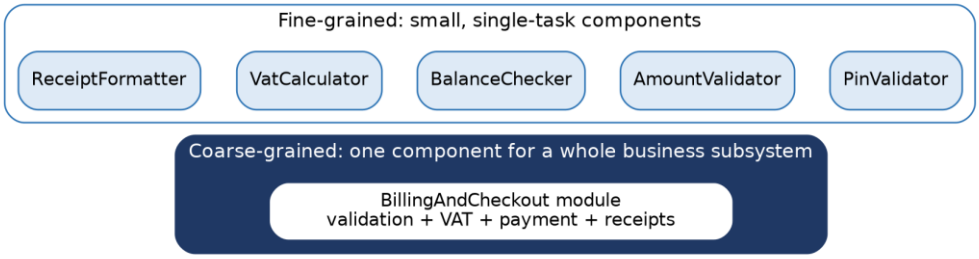
```
public class KraVatService : IVatService
{
    private const decimal Rate = 0.16m;

    public decimal CalculateVat(decimal netAmount)
    {
        if (netAmount < 0)                                          // enforce pre-condition
            throw new ArgumentException("Amount cannot be negative.");
        decimal vat = Math.Round(netAmount * Rate, 2);
        return vat;                                                 // satisfies post-conditi
on
    }
}
```

Code explained. The interface with XML documentation comments (`///`) is the specification: it tells every client what to pass in, what comes back and when an exception is thrown, without any code. `KraVatService` is one implementation. If the VAT rate changes, or the calculation moves to a KRA web service, only the implementation changes; clients written against `IVatService` keep working. Unit tests should test the **specification**, so any implementation can be checked against the same tests.

Component granularity

Granularity is the size and scope of a component.



Fine-grained components compared with a coarse-grained component

	Fine-grained	Coarse-grained
Size	Small, one specific task	Large, a whole business subsystem
Example	`PinValidator` class	`BillingAndCheckout` module
Advantages	Easy to test; highly reusable	Simpler high-level architecture; fewer interactions to manage
Disadvantages	High overhead managing interactions between hundreds of tiny components	Harder to reuse individual parts; larger to test and deploy

C# example

```
// Fine-grained: one small task
public class PinValidator
{
    public bool IsValid(string pin) =>
        pin.Length == 4 && pin.All(char.IsDigit);
}

// Coarse-grained: a whole subsystem behind one interface
public class BillingAndCheckout
{
    private readonly PinValidator pinValidator = new();
    private readonly KraVatService vat = new();
    private readonly ShoppingCart cart = new();

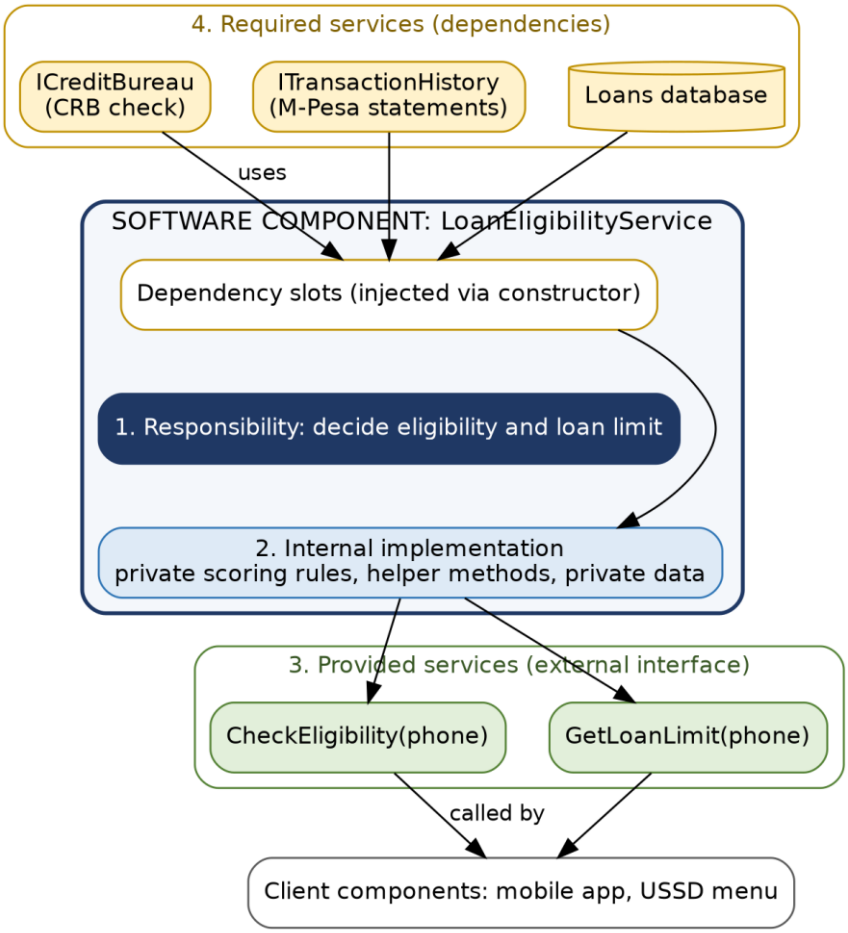
    public string Checkout(string pin, decimal netTotal)
    {
        if (!pinValidator.IsValid(pin)) return "Invalid PIN";
        decimal total = netTotal + vat.CalculateVat(netTotal);
        return $"Charged KES {total:N2}";
    }
}
```

Code explained. `PinValidator` does one thing: checks that a PIN has exactly four digits (`All(char.IsDigit)` needs `using System.Linq;`). `BillingAndCheckout` is coarse-grained: it hides a validator, VAT service and cart behind a single `Checkout` method. A client gets a simple interface, but cannot reuse only the VAT logic from this module.

> **Choosing granularity:** there is no single right size. Good guidelines are: group together things that change together; match components to business capabilities (payments, inventory, billing); and avoid components so small that the cost of connecting them exceeds their value.

2.2 Structure of a Software Component

Every component, however large or small, is made of the same five structural elements.



Structure of a software component, using a mobile-loan eligibility service as the example

1. **Component responsibilities:** the cohesive set of business tasks assigned to the component.
2. **Internal implementation:** source code, private data structures, algorithms and helper classes hidden behind the component wall.
3. **External services (provided services):** the public methods or API endpoints that other components can invoke.
4. **Dependencies (required services):** external components, APIs or databases that this component relies on to do its job.
5. **Component relationships:** how the component interacts with other components.

> **UML notation:** in UML component diagrams, a **provided interface** is drawn as a "lollipop" (a circle on a line) and a **required interface** as a "socket" (a half-circle). Two components connect when one component's socket fits another's lollipop.

C# example: all five elements in one component

Scenario: a Kenyan mobile-lending app decides whether a customer qualifies for a loan, based on their CRB status and M-Pesa transaction history.

```
// (4) REQUIRED SERVICES: what this component needs from others
public interface ICreditBureau
{
    bool IsListed(string phoneNumber);          // negatively listed on CRB?
}

public interface ITransactionHistory
{
    decimal AverageMonthlyInflow(string phoneNumber);
}

// (3) PROVIDED SERVICES: what this component offers to others
public interface ILoanEligibility
{
    bool CheckEligibility(string phoneNumber);
    decimal GetLoanLimit(string phoneNumber);
}

// (1) RESPONSIBILITY: decide eligibility and loan limit
public class LoanEligibilityService : ILoanEligibility
{
    // (4) dependency slots, filled from outside
    private readonly ICreditBureau crb;
    private readonly ITransactionHistory history;

    // (2) INTERNAL IMPLEMENTATION: private data
    private const decimal LimitPercentage = 0.30m;
    private const decimal MinimumInflow = 5000m;

    public LoanEligibilityService(ICreditBureau crb, ITransactionHistory history)
    {
        this.crb = crb;
        this.history = history;
    }

    public bool CheckEligibility(string phoneNumber) =>
        !crb.IsListed(phoneNumber) && history.AverageMonthlyInflow(phoneNumber) >= Minimum
Inflow;

    public decimal GetLoanLimit(string phoneNumber) =>
        CheckEligibility(phoneNumber) ? CalculateLimit(phoneNumber) : 0m;

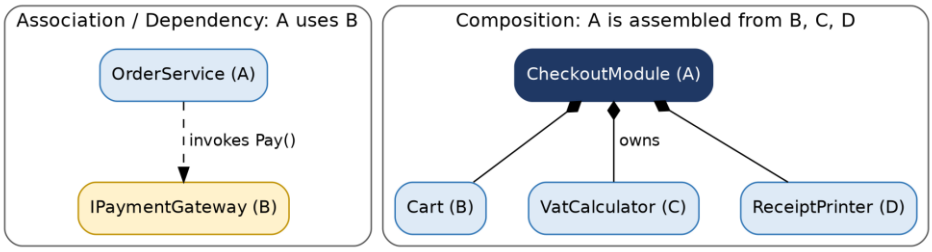
    // (2) INTERNAL IMPLEMENTATION: private helper
    private decimal CalculateLimit(string phoneNumber) =>
        Math.Floor(history.AverageMonthlyInflow(phoneNumber) * LimitPercentage / 100) * 10
0;
}
```

Code explained.

- **Required services (4):** `ICreditBureau` and `ITransactionHistory` describe what the component needs. It does not care whether the CRB check comes from TransUnion, Metropol or a test stub.
- **Provided services (3):** `ILoanEligibility` is the external interface the mobile app or USSD menu calls.
- **Responsibility (1):** the class does one cohesive job: eligibility and limits. It does not send SMS or disburse money.
- **Internal implementation (2):** the 30% rule, the KES 5,000 minimum and `CalculateLimit` are private. `Math.Floor(x / 100) * 100` rounds the limit down to the nearest hundred shillings. The lender can change the scoring rules without any client noticing.
- **Dependencies are injected** through the constructor, so the component is loosely coupled and easy to test.

For a customer with an average monthly inflow of KES 23,000 and no CRB listing, `GetLoanLimit` returns KES 6,900.

Component relationships



Dependency compared with composition

- **Association / dependency:** component A invokes services of component B. B exists independently of A and may be shared by many components.
- **Composition:** component A is built by assembling smaller components B, C and D. A owns them, and they exist as parts of A.

	Dependency	Composition
Meaning	A **uses** B	A **is made of** B, C, D
Lifetime	B lives independently of A	Parts are created and managed as part of A
Coupling	Loose, through an interface	Tighter; parts are internal to A
UML symbol	Dashed arrow	Filled diamond on A's side

```
// Dependency: OrderService USES a payment gateway supplied from outside
public class OrderService
{
    private readonly IPaymentGateway gateway;
    public OrderService(IPaymentGateway gateway) => this.gateway = gateway;
}

// Composition: CheckoutModule IS BUILT FROM its parts
public class CheckoutModule
{
    private readonly ShoppingCart cart = new();        // part B
    private readonly VatCalculator vat = new();        // part C
    private readonly ReceiptPrinter printer = new();   // part D
}
```

Code explained. `OrderService` receives its gateway from outside; the same gateway object could serve many services, so this is a dependency. `CheckoutModule` creates its own cart, VAT calculator and printer; they exist only as parts of the checkout, so this is composition. In real systems, even composed parts are often injected through interfaces to keep them replaceable, as section 2.5 shows.

Summary: linking fundamentals to characteristics

Sections 2.1 and 2.2 described **what a component is** and **what it is made of**. The next section asks a different question: **what qualities make a component good?** Each characteristic in 2.3 grows from a fundamental idea:

Fundamental idea (2.1 and 2.2)	Leads to characteristic (2.3)
Distinct responsibility, high cohesion	Localized responsibilities
Boundary and hidden implementation	Independence and loose coupling
Independently deployable unit	Deployability and versioning
Provided and required services	Composability
Specification separate from implementation	Reusability and replaceability

2.3 Software Component Characteristics

Building on the definition and structure in 2.1 and 2.2, a well-designed component shows four groups of characteristics: independence, deployability, composability, and reusability with replaceability.

Analogy: the matatu spare part. A matatu is built from parts: engine, gearbox, tyres, radio. When a tyre bursts on Thika Road, nobody rebuilds the vehicle. Any tyre with the right size and bolt pattern fits. The tyre has a standard interface, does one job, and can be swapped without touching the engine.

Independence

- **Limited dependency:** a component relies only on the few components it truly needs. An SMS component should not depend on the inventory database.
- **Loose coupling:** components talk through agreed interfaces, not internal details. A shop needs only its M-Pesa till number; Safaricom can upgrade its servers and the shop keeps working.
- **Localized responsibilities:** one clear area of responsibility per component. A payment bug is found in the payment component, nowhere else.

Deployability

- **Packaging:** in .NET, components ship as assemblies (.dll), NuGet packages, or at the architectural level as containers and web services.
- **Independent deployment:** release a new version of one component without redeploying the whole system.
- **Versioning and replacement:** semantic versioning MAJOR.MINOR.PATCH. MAJOR = interface broke, MINOR = compatible new features, PATCH = bug fixes.

Composability

- **Combining components:** larger systems are assembled from parts, like a house from bricks, doors and windows from different suppliers.
- **Dependencies:** when A uses B, A depends on B. Dependencies should be declared openly, for example in a constructor.
- **Composition constraints:** components fit only if interfaces, data formats, versions and protocols match. A Kenyan 3-pin plug needs an adapter for a 2-pin socket; in software an adapter component plays that role.

Reusability and replaceability

Reusability: one M-Pesa payment component can serve a school fees portal, an online shop and a utility billing system. **Replaceability:** an email notifier can be swapped for an SMS notifier without changing the code that sends notifications.

C# example

```
// Public contract: what the outside world sees
public interface ISmsSender
{
    bool Send(string phoneNumber, string message);
}

// One implementation of the contract
public class SafaricomSmsSender : ISmsSender
{
    public bool Send(string phoneNumber, string message)
    {
        // Details of calling the SMS gateway are hidden here
        Console.WriteLine($"[Safaricom SMS] To {phoneNumber}: {message}");
        return true;
```

```
    }
}

// A replacement implementation with the same contract
public class AirtelSmsSender : ISmsSender
{
    public bool Send(string phoneNumber, string message)
    {
        Console.WriteLine($"[Airtel SMS] To {phoneNumber}: {message}");
        return true;
    }
}
```

Code explained. `ISmsSender` declares that every SMS component must offer `Send`, returning true or false; it holds no implementation. The `: ISmsSender` syntax means "implements". Each class supplies its own `Send`; here they print to the console, but a real one would call its provider's gateway. The example shows localized responsibility (SMS only), replaceability (either class fits anywhere `ISmsSender` is expected), reusability, and deployability if packaged as its own .dll.

Activity: identify the characteristics

Give students this class and ask which characteristics are present or missing.

```
public class ReportGenerator
{
    public void GenerateSalesReport()
    {
        var conn = new SqlConnection("Server=.;Database=Shop;...");
        conn.Open();
        // query sales, format as PDF, email to manager
        var smtp = new SmtpClient("smtp.gmail.com");
        // send email...
    }
}
```

Model answer. Responsibility is not localized: it queries the database, builds a PDF and sends email. It is tightly coupled: it creates `SqlConnection` and `SmtpClient` itself with hard-coded settings. It is not replaceable: with no interface, switching email providers means editing the class. Reusability is low: nobody can reuse just the PDF part. Better design: split into `ISalesRepository`, `IReportFormatter` and `IEmailSender`, each injected into the generator.

2.4 Public Interfaces and Hidden Implementation (40 min)

Every component has two faces: a public view clients use, and an internal implementation they never see.

- **Public view:** method names, parameters, return types and the promised behaviour.
- **Internal implementation:** private fields, helper methods, algorithms, databases, files, third-party libraries.

Analogy: the M-Pesa agent. Withdrawing cash in Kawangware, you give your phone number, ID and amount, and receive cash. You never see float management, transaction validation, ledger updates or fraud checks. Safaricom can rebuild its backend and your experience at the counter stays the same.

Key ideas

- **Information hiding** (Parnas, 1972): hide the design decisions most likely to change, such as file formats, databases and external APIs, behind a stable interface.
- **Encapsulation:** bundle data with the methods that use it and restrict direct access. C# access modifiers: `public`, `private`, `protected`, `internal` (same assembly only, useful for component-level hiding).
- **Services exposed:** operations the component promises, e.g. log a message, process a payment, check stock.
- **Hidden details:** where logs are stored, which payment API is called, table names, retries, caching.
- **Why implementation independence matters:** bugs get fixed, performance improves or technology changes without breaking clients; teams can work in parallel against the interface.
- **Programming to interfaces:** code refers to the contract (`ILogger`), not the concrete class (`FileLogger`). The concrete class is chosen in one place only.

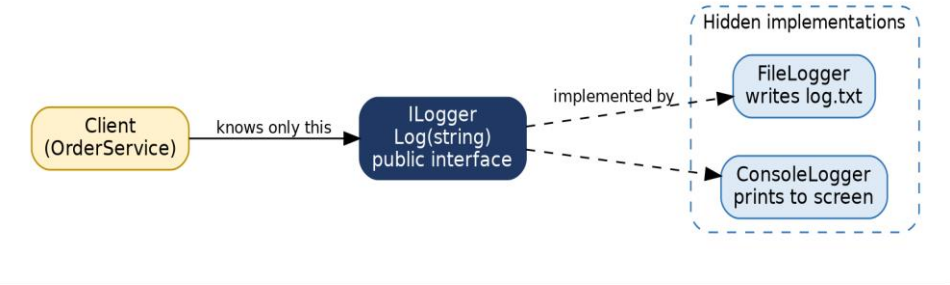
C# example: the logger

```
using System.IO;

public interface ILogger
{
    void Log(string message);
}

public class FileLogger : ILogger
{
    public void Log(string message)
    {
        // Internal implementation
        File.AppendAllText("log.txt", message);
    }
}
```

Code explained. `ILogger` is the public contract: a logger can `Log` a message, nothing about files or clouds. `FileLogger` fulfils it with `File.AppendAllText`, which opens log.txt (creating it if missing), appends the message and closes the file. That file handling is the hidden implementation.



The client depends only on the ILogger interface; implementations stay hidden

The client knows only `Log(string)`: not the file name, location, format, or that a file is used at all. A second implementation proves it:

```
public class ConsoleLogger : ILogger
{
    public void Log(string message)
    {
        Console.WriteLine($"{DateTime.Now:yyyy-MM-dd HH:mm:ss} {message}");
    }
}

// Client code
ILogger logger = new ConsoleLogger();   // could equally be new FileLogger()
logger.Log("M-Pesa payment received: KES 1,500");
```

Code explained. `logger` is declared with the interface type. Only `new ConsoleLogger()` names a concrete class; change that word to `FileLogger` and the whole logging behaviour switches, while `logger.Log(...)` stays identical.

Encapsulation example: MpesaWallet

```
public class MpesaWallet
{
    // Hidden state: nobody outside can change this directly
    private decimal balance;
    private readonly List<string> transactionHistory = new();

    // Public service: read-only view of the balance
    public decimal Balance => balance;

    public void Deposit(decimal amount)
    {
        ValidateAmount(amount);
        balance += amount;
        Record($"Deposit KES {amount}");
    }

    public bool Withdraw(decimal amount)
    {
        ValidateAmount(amount);
        if (amount > balance) return false;
        balance -= amount;
        Record($"Withdraw KES {amount}");
        return true;
    }

    // Hidden helper methods
    private void ValidateAmount(decimal amount)
    {
```

```
        if (amount <= 0)
            throw new ArgumentException("Amount must be positive.");
    }

    private void Record(string entry)
    {
        transactionHistory.Add($"{DateTime.Now}: {entry}");
    }
}
```

Code explained. `balance` is private, so `wallet.balance = 1000000;` will not compile. It changes only through `Deposit` and `Withdraw`, which enforce the rules: positive amounts, no overdraw. `Balance => balance` lets clients read but not set the balance. `ValidateAmount` and `Record` are private machinery.

Activity: public part vs hidden implementation

Students fill in this table for `MpesaWallet`.

Public (clients can use)	Hidden (implementation)
`Balance` property, read only	`balance` field
`Deposit(decimal)`	`transactionHistory` list
`Withdraw(decimal)` returning bool	`ValidateAmount()` rules
	`Record()` and its date format

Follow-up: if history moves from a list to a SQL database, which clients must change? None, because history was never public.

2.5 Component Composition Principles

Composition builds a larger system by connecting components through their defined services, with each component depending on abstractions rather than concrete classes.

Analogy: the wedding committee. A Kenyan wedding is composed of providers: caterer, tent hire, DJ, photographer. The committee links them through agreed contracts (date, venue, budget, menu). If the DJ cancels, another DJ meeting the same contract steps in and the wedding carries on. The committee is the composer; the contract is the interface.

- **Connecting through services:** the caller knows what service is offered, not who provides it or how.
- **Dependencies:** make them explicit and point them at interfaces.
- **Compatibility:** the required interface must match the provided one: signatures, data types, behaviour (does failure throw or return false?), versions.
- **Loose coupling:** the component receives its dependency from outside instead of creating it. This is **dependency injection**; the common form is constructor injection.

C# example: OrderService

```
public class OrderService
{
    private readonly ILogger logger;

    public OrderService(ILogger logger)
    {
        this.logger = logger;
    }

    public void PlaceOrder(string customer, decimal total)
    {
        // business logic for placing an order...
        logger.Log($"Order placed by {customer} for KES {total}");
    }
}
```

Code explained. The field has the interface type `ILogger`; `readonly` means it is set only in the constructor and cannot be swapped by accident. The constructor forces whoever creates an `OrderService` to supply a logger, so the dependency is explicit. `this.logger` separates the field from the same-named parameter. `PlaceOrder` logs through the interface without knowing the destination. **OrderService depends on an abstraction, not a concrete implementation.**

Replacing one implementation with another

```
// Development
var devService = new OrderService(new ConsoleLogger());
devService.PlaceOrder("Wanjiku", 2500m);

// Production
var prodService = new OrderService(new FileLogger());
prodService.PlaceOrder("Otieno", 4800m);
```

Code explained. The same class is composed with two loggers; nothing inside `OrderService` changed. The `m` suffix makes the numbers `decimal`, the correct type for money.

Contrast: tight coupling

```
public class TightOrderService
{
```

```
    private readonly FileLogger logger = new FileLogger(); // concrete, created inside

    public void PlaceOrder(string customer, decimal total)
    {
        logger.Log($"Order by {customer}");
    }
}
```

Switching to console or cloud logging now means editing and redeploying this class, and it cannot be tested without writing a real file.

Composition in ASP.NET Core

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSingleton<ILogger, FileLogger>();
builder.Services.AddScoped<OrderService>();

var app = builder.Build();
```

Code explained. `AddSingleton<ILogger, FileLogger>()` says: whenever an `ILogger` is requested, supply one shared `FileLogger`. `AddScoped<OrderService>()` creates one order service per web request. .NET reads the constructor, sees it needs an `ILogger`, and injects it. Changing the logger system-wide is one line. Note: .NET has its own `ILogger` in Microsoft.Extensions.Logging, so a real project would name ours `IAppLogger`.

2.6 Taxonomy of Component Models

Components exist at two broad levels: object-level classes inside one program, and architectural-level services that are larger, API-driven and often deployed on their own.

Analogy: the Huduma Centre. Object-level components are individual staff: clerk, cashier, verifier, each with one job. An architectural component is a whole desk such as SHA services, which the public uses as one unit through a counter, however many staff work behind it.

A. Object-level components

Represented by classes and objects; encapsulate state (fields) and behaviour (methods); interact through method calls defined by interfaces.

```
public interface IPayment
{
    void Pay(decimal amount);
}

public class MpesaPayment : IPayment
{
    private readonly string phoneNumber;

    public MpesaPayment(string phoneNumber)
    {
        this.phoneNumber = phoneNumber;
    }

    public void Pay(decimal amount)
    {
        // Hidden: would send an STK push request to the M-Pesa API
        Console.WriteLine($"STK push sent to {phoneNumber} for KES {amount}");
    }
}

public class CardPayment : IPayment
{
    private readonly string cardToken;

    public CardPayment(string cardToken)
    {
        this.cardToken = cardToken;
    }

    public void Pay(decimal amount)
    {
        // Hidden: would call a card payment gateway
        Console.WriteLine($"Card charged KES {amount}");
    }
}
```

Code explained. `IPayment` defines one service, `Pay`. `MpesaPayment` keeps the phone number as private state and would trigger an STK push, the phone pop-up asking for the M-Pesa PIN. `CardPayment` keeps a card token instead. Both live in the same program, are created with `new`, and are used through method calls; a checkout class can accept any `IPayment`.

B. Architectural-level components

Larger units: services, modules, subsystems, deployable applications. One is built from many classes and communicates through an API (HTTP/REST, gRPC, message queues). Examples: Payment Service, Authentication Service, Inventory Service.

```
// Payment Service: a separate ASP.NET Core web application
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton<IPaymentProcessor, MpesaProcessor>();
var app = builder.Build();

app.MapPost("/api/payments", (PaymentRequest request, IPaymentProcessor processor) =>
{
    var result = processor.Process(request.PhoneNumber, request.Amount);
    return result.Success ? Results.Ok(result) : Results.BadRequest(result);
});

app.Run();

public record PaymentRequest(string PhoneNumber, decimal Amount);
public record PaymentResult(bool Success, string Reference);

public interface IPaymentProcessor
{
    PaymentResult Process(string phoneNumber, decimal amount);
}

public class MpesaProcessor : IPaymentProcessor
{
    public PaymentResult Process(string phoneNumber, decimal amount)
    {
        // Hidden: call M-Pesa, store transaction, check fraud rules...
        return new PaymentResult(true, $"MP{DateTime.Now:yyyyMMddHHmmss}");
    }
}
```

Code explained. This is a whole web application. `MapPost` creates an HTTP endpoint; any system can POST JSON such as `{"phoneNumber":"0712345678","amount":1500}`. .NET turns the JSON into a `PaymentRequest` record and injects the registered processor. The endpoint returns HTTP 200 on success or 400 on failure. `record` is a concise C# data type. Callers know only the URL and JSON format, not the language, provider or database. The service can be deployed, scaled and updated on its own server.

Object vs architectural component

Aspect	Object-level	Architectural-level
Size	Usually smaller	Usually larger
Orientation	Class/object	System/architecture
Encapsulates	State and behaviour	A subsystem or service
Interaction	Method calls	API calls (HTTP, messages)
Deployment	With the application	Often independently
Example	PaymentProcessor class	Payment Service

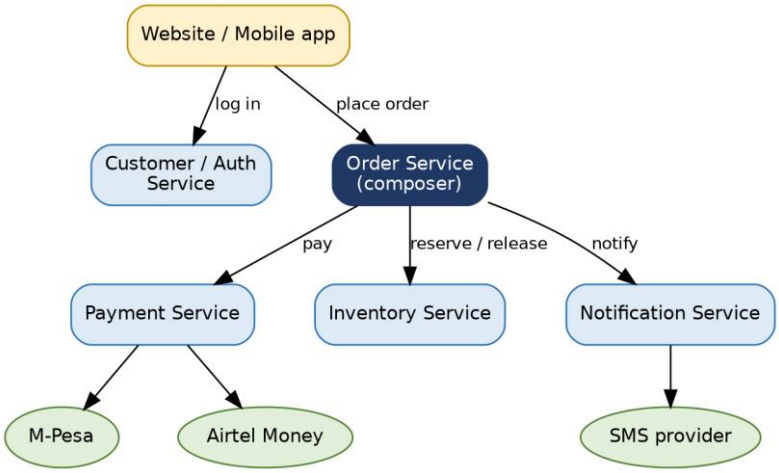
2.7 Case Study: DukaBora Online, Nairobi

DukaBora, a fictional Nairobi online shop, fixed a fragile all-in-one checkout by splitting it into Order, Payment, Inventory and Notification components joined through interfaces.

Background. DukaBora sells electronics and household goods across Kenya. Customers order via website and app, pay by M-Pesa, and collect from riders or pickup points in Nairobi, Mombasa, Kisumu and Eldoret.



Order flow in the online shop



DukaBora after the redesign: independent services composed by the Order Service

The problem

The first system was one large application, with checkout written like this:

```
public class Checkout
{
    public void Complete(string phone, int productId, int qty)
    {
        // 1. check stock directly in SQL
        // 2. call M-Pesa API directly with hard-coded keys
        // 3. reduce stock in SQL
        // 4. send SMS directly through a provider's API
        // 5. write to log.txt
    }
}
```

1. Adding Airtel Money and cards meant editing `Checkout`, which also held stock and SMS code; a payment mistake broke stock updates.
2. On Black Friday, payment callbacks and browsing shared one app, so the whole site slowed and the busy part could not be scaled on its own.
3. Changed M-Pesa credentials forced a full redeployment, taking the shop offline for browsers too.
4. Some customers were charged but no order was recorded, because payment and order logic were tangled.

5. Testing needed real M-Pesa calls.

The solution

Payment, Inventory, Orders and Notifications become separate architectural-level services. Inside each, object-level classes implement interfaces so providers can be swapped.

Task 1: identify possible components

Component	Responsibility
Customer / Authentication Service	Register, log in, manage profiles and addresses
Order Service	Create orders, track status: Pending, Paid, Shipped, Delivered
Payment Service	M-Pesa, Airtel Money, card payments and callbacks
Inventory Service	Stock per warehouse; reserve and release stock
Notification Service	SMS and email confirmations

Task 2: classify

Architectural-level: Customer, Order, Payment, Inventory and Notification Services, each with its own API and data. **Object-level:** the classes inside them, such as `Order`, `OrderItem`, `Customer`, `MpesaPayment`, `AirtelMoneyPayment`, `StockItem`, `SmsSender`.

Task 3: independently deployable?

Component	Deploy independently?	Why
Payment Service	Yes	Providers and API versions change; needs strong security and own scaling during sales
Inventory Service	Yes	Warehouses and suppliers update stock on their own schedule
Order Service	Yes	Core flow; changes with business rules
Notification Service	Yes	SMS providers and templates change; failures must never block orders
Customer / Auth Service	Yes	Security isolation
Object-level classes	No	Ship inside their service

Tasks 4 and 5: public vs hidden

Component	Public	Hidden
Payment	Start payment (phone, amount, order ID); check status; result with reference	M-Pesa keys and secrets, STK push format, callback handling, retries, fraud rules, tables
Inventory	Check availability; reserve; release	Which warehouse, database schema, reservation timeouts
Order	Place order; get status; cancel	Status transitions, totals and delivery fee calculation
Notification	Send notification (recipient, message)	SMS provider, sender IDs, API keys

Task 6: composition, full C# solution

A runnable single-program version; each region becomes a separate service in production.

```
using System;
using System.Collections.Generic;

// ================ PUBLIC CONTRACTS ================
```

```
public record PaymentResult(bool Success, string Reference, string Message);

public interface IPaymentGateway
{
    PaymentResult Pay(string phoneNumber, decimal amount, string orderId);
}

public interface IInventory
{
    bool Reserve(string productCode, int quantity);
    void Release(string productCode, int quantity);
}

public interface INotifier
{
    void Notify(string phoneNumber, string message);
}

// ================ PAYMENT COMPONENT (hidden details) ================
public class MpesaGateway : IPaymentGateway
{
    private const string ShortCode = "174379";          // hidden
    private readonly Random random = new();              // simulates API response

    public PaymentResult Pay(string phoneNumber, decimal amount, string orderId)
    {
        // In reality: get OAuth token, send STK push, wait for callback
        bool customerEnteredPin = random.Next(0, 10) > 1;  // ~80% success
        return customerEnteredPin
            ? new PaymentResult(true, $"MPESA-{orderId}", "Payment received")
            : new PaymentResult(false, "", "Customer cancelled or timed out");
    }
}

public class AirtelMoneyGateway : IPaymentGateway
{
    public PaymentResult Pay(string phoneNumber, decimal amount, string orderId)
    {
        // Different API, same contract
        return new PaymentResult(true, $"AIRTEL-{orderId}", "Payment received");
    }
}

// ================ INVENTORY COMPONENT ================
public class WarehouseInventory : IInventory
{
    private readonly Dictionary<string, int> stock = new()   // hidden storage
    {
        ["TV-43"] = 5,
        ["FRIDGE-200L"] = 2
    };

    public bool Reserve(string productCode, int quantity)
    {
        if (!stock.TryGetValue(productCode, out int available) || available < quantity)
            return false;
        stock[productCode] = available - quantity;
        return true;
    }

    public void Release(string productCode, int quantity)
    {
        if (stock.ContainsKey(productCode))
            stock[productCode] += quantity;
```

```
    }
}

// ================ NOTIFICATION COMPONENT ================
public class SmsNotifier : INotifier
{
    public void Notify(string phoneNumber, string message)
    {
        Console.WriteLine($"SMS to {phoneNumber}: {message}");
    }
}

// ================ ORDER COMPONENT (the composer) ================
public class OrderService
{
    private readonly IInventory inventory;
    private readonly IPaymentGateway payment;
    private readonly INotifier notifier;

    public OrderService(IInventory inventory, IPaymentGateway payment, INotifier notifier)
    {
        this.inventory = inventory;
        this.payment = payment;
        this.notifier = notifier;
    }

    public bool PlaceOrder(string phone, string productCode, int qty, decimal price)
    {
        string orderId = Guid.NewGuid().ToString("N")[..8].ToUpper();
        decimal total = price * qty;

        // Step 1: reserve stock
        if (!inventory.Reserve(productCode, qty))
        {
            notifier.Notify(phone, $"Sorry, {productCode} is out of stock.");
            return false;
        }

        // Step 2: take payment
        PaymentResult result = payment.Pay(phone, total, orderId);

        // Step 3: if payment fails, give the stock back
        if (!result.Success)
        {
            inventory.Release(productCode, qty);
            notifier.Notify(phone, $"Payment failed: {result.Message}. Order {orderId} can
celled.");
            return false;
        }

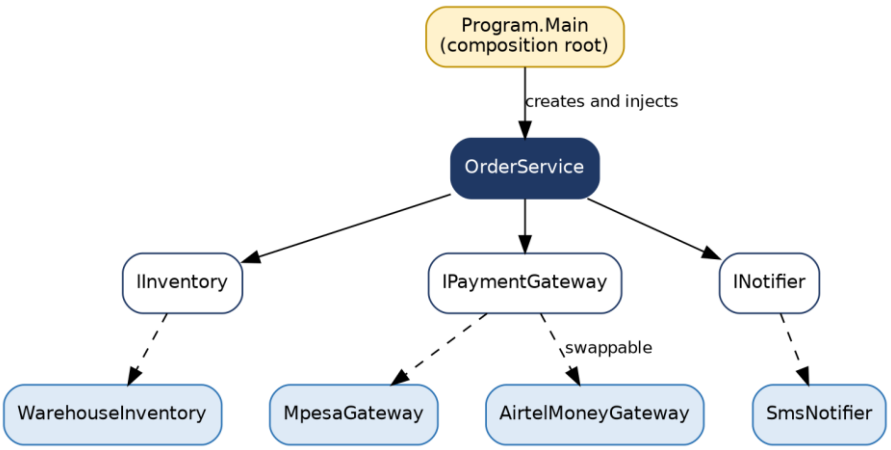
        // Step 4: confirm
        notifier.Notify(phone, $"Order {orderId} confirmed. KES {total:N0} paid. Ref {resu
lt.Reference}.");
        return true;
    }
}

// ================ COMPOSITION ROOT ================
public class Program
{
    public static void Main()
    {
        IInventory inventory = new WarehouseInventory();
        INotifier notifier = new SmsNotifier();
```

```
        // Customer chooses M-Pesa
        var mpesaOrders = new OrderService(inventory, new MpesaGateway(), notifier);
        mpesaOrders.PlaceOrder("0712345678", "TV-43", 1, 32000m);

        // Customer chooses Airtel Money: only the gateway changes
        var airtelOrders = new OrderService(inventory, new AirtelMoneyGateway(), notifier)
;
        airtelOrders.PlaceOrder("0733123456", "FRIDGE-200L", 1, 45000m);

        // Out of stock example
        airtelOrders.PlaceOrder("0733123456", "FRIDGE-200L", 5, 45000m);
    }
}
```



Composition root wiring the DukaBora components

Code explained

- **Public contracts:** the three interfaces plus `PaymentResult` are all components know of each other. `PaymentResult` carries success, reference and message so callers react properly.
- **Payment:** `MpesaGateway` hides its short code (`private const`) and the M-Pesa process. `Random` simulates the roughly 20% of customers who ignore the STK push or mistype the PIN. `AirtelMoneyGateway` implements the same interface, showing replaceability.
- **Inventory:** stock sits in a private `Dictionary<string, int>`. `Reserve` uses `TryGetValue` to check the product exists and has enough, then subtracts; `Release` adds back. It could move to SQL with no change to `OrderService`.
- **Notification:** `SmsNotifier` hides delivery; email or WhatsApp is just a new `INotifier`.
- **Order (the composer):** receives all dependencies by constructor injection. `Guid.NewGuid().ToString("N")[..8]` makes a short order ID. Stock is reserved first so two buyers cannot take the last TV; if payment fails, stock is released. `{total:N0}` prints 32,000 with separators.
- **Composition root:** `Main` is the only place naming concrete classes. M-Pesa to Airtel Money is a one-word change, and both order services share one inventory and notifier.

How the design solves DukaBora's problems

Original problem	How the component design solves it
New payment methods broke other code	New gateways implement `IPaymentGateway`; order and inventory code untouched

Original problem	How the component design solves it
Could not scale on Black Friday	Payment, Inventory and Orders scale as separate services
M-Pesa changes forced full redeployment	Only the Payment Service is redeployed
Charged but no order recorded	Clear flow with `PaymentResult` and stock release on failure
Tests needed real M-Pesa calls	A `FakePaymentGateway` is injected in tests

Discussion questions

1. If Safaricom changes the STK push API tomorrow, which components change and which stay untouched?
2. If the SMS provider is down, should the order still succeed? Change `OrderService` so notification failures never block an order.
3. Write a `FakePaymentGateway` that always fails and use it to test that stock is released.
4. DukaBora adds a Delivery Service for boda boda riders. Where is the boundary, and what is its public interface?

Key Terms

Term	Meaning
Component	Modular, encapsulated, independently deployable unit with a distinct function and well-defined interfaces
Interface / specification	The contract: inputs, outputs, exceptions, pre- and post-conditions
Implementation	The hidden code, data and algorithms that fulfil the specification
Boundary	The line separating the public contract from hidden internals
Granularity	The size and scope of a component: fine-grained or coarse-grained
Provided service	An operation a component offers to others
Required service	An operation a component needs from others (a dependency)
Cohesion	How closely a component's responsibilities belong together (aim high)
Coupling	How much components depend on each other's details (aim low)
Dependency injection	Supplying a component's dependencies from outside, usually through its constructor
Composition root	The single place where concrete components are created and wired together
Object-level component	A class or object interacting through method calls
Architectural-level component	A service or subsystem interacting through an API, often independently deployed